

IFN647 Workshop (Week 2)

Pre-processing Textual Data

.....

Objectives:

- Getting familiar with Python for solving practical tasks: Understand basic Python concepts (see week 1 Python tutorial), and learn how to edit and run .py and .ipynb files using Pycharm, IDLE, and Anaconda.
- Understand how to use Python for text data pre-processing.

Please note that this unit assumes prior experience with programming, as well as foundational knowledge of machine learning and data mining. If you encounter any difficulties using Python, please contact the unit coordinator or your tutor, as Python will be required for workshop activities, lecture review exercises, and IFN647 assessment questions.

Task 1: Write a program that loads (reads) an XML document, and prints out the **itemid** and the number of words in **<text>** of the document.

The following is the Format of the Document:

```
<newsitem xml:lang="en" date="1997-07-20" id="root" itemid="741299">
<title>BELGIUM: MOTOR RACING-LEHTO AND SOPER HOLD ON FOR GT
VICTORY.</title>
<headline>MOTOR RACING-LEHTO AND SOPER HOLD ON FOR GT
VICTORY.</headline>
<dateline>SPA FRANCORCHAMPS, Belgium</dateline>
<text>
<p>J.J. Lehto of Finland and Steve Soper of Britain drove their ailing
McLaren to victory in the fifth round of the world GT championship on
Sunday, beating the Mercedes of German Bernd Schneider and Austrian
Alexander Wurz by 15 seconds.</p>
```

```
<p>Their victory enabled them to open up a 16-point lead in the overall
standings over Schneider, who mounted a strong challenge on the
struggling leaders in the final minutes of the four-hour race.</p>
<p>But Soper, struggling with the car's handling caused by a broken
undertray, just managed to hold on for the win.</p>
<p>Lehto had opened up a lead of over 90 seconds during a mid-race
downpour in the Ardennes mountains.</p>
<p>"I thought that everyone else was driving on dry-weather tyres," he
joked afterwards.</p>
<p>"We swapped to rain tyres at exactly the right time and I was able
to push hard and open up a big lead."</p>
<p>Third to finish was the Porsche of France's Bob Wollek and Yannick
Dalmas and Belgian Thierry Boutsen.</p>
<p>The Belgian, a former Formula One driver, switched from the car he
normally shares with German Hans Stuck following a power-steering
failure on his own car.</p>
</text>
<copyright>(c) Reuters Limited 1997</copyright>
</newsitem>
```

Task 2: Design a parsing function (`parse_doc(input, stops)`) to read a file and represent the file as a tuple

$$(word_count, \{docid:curr_doc\})$$

where

- `word_count` is the number of words in `<text> ... <\text>`
- `docid` is simply assigned by the `'itemid'` in `<newsitem>`
- `curr_doc` is a dictionary of term_frequency pairs.

You only need to tokenize the `'<text>...</text>'` part of the document into words, exclude all tags, and discard punctuations and numbers. Then please remove stopping words and at last get all terms used in the `'<text>...</text>'` part.

- Download the stopping words list (`common-english-words.txt`) from the Canvas, and use it for this task.

You can initialize dictionary `curr_doc = {}`, then add terms into `curr_doc = {}`, when you go through terms, you may need

to check if the new term exists in `curr_doc` and then update its frequency.

The following is an example of the return value of `parse_doc()` for file "6146.xml" (see the file in the data folder in the Canvas):

```
(133, {'6146': {'argentine': 1, 'bonds': 1, 'slightly': 1, 'higher': 1, 'small': 1, 'technical': 2, 'bounce': 2, ... , 'newsroom': 1}})
```

Task 3: Design a main function to read a xml file and `common-english-words.txt` (the list of stopping words), call function `parse_doc(input, stops)`, and print the `itemid` (`docid`), the number of words (`word_count`) and the number of terms (`len(curr_doc)`).

The following is an example of the outputs:

Document itemid: 6146 contains: 133 words and 75 terms